FPT UNIVERSITY

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

Lightweight Deep Learning for Robust Shrimp Disease Detection on Mobile Devices under Diverse Environmental Conditions

CVio	
Group Members	Tran Huu Nhan (Leader) - CE181655 Le Thi Huynh Nhu - CA182007 Nguyen Van Phong - CE192081
Supervisor	Nguyen Dinh Vinh
Ext Supervisor	N/A
Capstone Project code	AIP491_01

- Can Tho, 08/2026 -

Table of Contents

Acknowledgement

The team sincerely thanks Nguyen Dinh Vinh, Lecturer and Supervisor, for his academic guidance, milestone reviews, and detailed feedback throughout the capstone project. We acknowledge FPT University Can Tho for the academic environment and project framework. We also acknowledge the authors and maintainers of the public shrimp-image datasets, open-source frameworks, and research tools used in the classification, instance-segmentation, robustness, explainability, and mobile application work. Finally, the team thanks the lecturers, colleagues, friends, and families who supported experimentation, application development, documentation, presentation, and defense preparation.

Definition and Acronyms

The following terms are used throughout this standalone report. The list is intentionally limited to terminology that appears in Report 3.

Acronym	Definition
AI	Artificial Intelligence
ASL	Asymmetric Loss
BG	Black Gill disease
CAM	Class Activation Mapping
CBAM	Convolutional Block Attention Module
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DCFR	Project-created texture feature-recalibration component used with SimAM
ECA	Efficient Channel Attention
FCN	Fully Convolutional Network
FGSM	Fast Gradient Sign Method
Grad-CAM	Gradient-weighted Class Activation Mapping
Grad-CAM++	Generalized gradient-based class activation mapping
GRU	Gated Recurrent Unit
IHHNV	Infectious Hypodermal and Hematopoietic Necrosis Virus
IoT	Internet of Things
KNN	K-Nearest Neighbors
LDAM	Label-Distribution-Aware Margin
NMS	Non-Maximum Suppression
ROI	Region of Interest
SimAM	Simple, parameter-free attention module based on neuron energy
TIMM	PyTorch Image Models library
WSSV	White Spot Syndrome Virus
XAI	Explainable Artificial Intelligence
YHV	Yellow Head Virus
YOLO	You Only Look Once model family

List of tables

List of figures

No figures are included in this standalone report.

III. Existing Systems/State of the Art

1. Overview of the Field

Shrimp aquaculture has repeatedly been affected by viral, bacterial, parasitic, and environmentally mediated disease events. Historical reviews describe how pathogens such as WSSV, IHHNV, YHV, and related syndromes caused severe production losses and forced the industry to adopt stronger biosecurity, surveillance, and health-management practices [1–9]. These studies establish the practical motivation for rapid screening, but they also show why image analysis cannot replace laboratory confirmation: multiple diseases can share visible symptoms, signs may appear only at particular infection stages, and environmental stress can produce visually similar abnormalities.

Computer vision provides a complementary, non-invasive route for preliminary screening. In the broader smart-aquaculture literature, deep learning has been applied to species recognition, biomass estimation, behavior monitoring, water-quality support, and disease-oriented image analysis [10]. For shrimp disease research, the most relevant tasks are image classification and instance segmentation. Classification assigns an image-level label and is appropriate for fast mobile screening. Instance segmentation predicts a separate mask for each visible disease region and therefore supports spatial evidence, stricter localization evaluation, and investigation of small or irregular lesions.

The literature is fragmented across several technical directions. Shrimp-specific studies often optimize clean-test classification accuracy on a single dataset, whereas general computer-vision research separately advances efficient backbones, imbalanced learning, attention, corruption robustness, explainable AI, instance segmentation, and mobile deployment. A defensible state-of-the-art review must therefore connect these directions without treating metrics from different datasets as directly rankable. Reported results are meaningful only together with the dataset, class definitions, preprocessing, split policy, checkpoint selection, and evaluation metric used by the original study.

This chapter follows a chronological and analytical structure. It first traces the transition from biological disease management to image-based screening, then examines the theories and systems that support lightweight classification and mask-level localization. The comparison tables preserve results and claims from the reviewed publications while explicitly identifying limitations relative to the thesis. Project-specific proposed modules and experimental results are not presented as existing methods; they are introduced only in the later Methodology and Results chapters.

2. Historical Context

2.1 Biological and Conventional Diagnostic Foundations

Before deep learning became relevant, shrimp disease research concentrated on pathogen emergence, transmission, clinical signs, histopathology, molecular diagnosis, and farm-level control. Studies from the late 1990s onward documented the destructive effect of viral diseases on penaeid shrimp and emphasized exclusion, pathogen-free stocks, movement control, and laboratory testing [1], [2]. Later reviews connected recurrent disease emergence with the rapid globalization and intensification of shrimp production [3], [4].

By 2012, the literature had shifted from describing individual outbreaks toward a system-level view of food security and pathogen dissemination. Stentiford et al. argued that disease could constrain future crustacean supply, while Flegel and Seibert and Pinto reviewed the emergence, distribution, and biology of major shrimp pathogens [5–7]. Thitamadee et al. subsequently summarized important disease threats facing cultivated penaeid shrimp in Asia, including syndromes whose visible signs can overlap with environmental stress or secondary infection [8]. This history explains why an image model should be framed as preliminary screening rather than autonomous diagnosis.

2.2 Early Image Analysis and Transfer Learning

The first practical transition toward data-driven image recognition occurred when convolutional neural networks became transferable to small domain datasets. Duong-Trung et al. demonstrated that pre-trained CNNs could classify multiple shrimp disease categories, establishing an early field-image baseline for this topic [11]. The study was important because it reduced dependence on handcrafted visual descriptors, but it remained centered on image-level prediction and did not provide disease-region masks, mobile deployment evidence, or specimen-grouped leakage analysis.

A second stage emphasized compact custom architectures. Wang et al. modified LeNet for rapid classification of hepatopancreatic necrosis, red-body, and whitish-muscle disease in Penaeus vannamei [12]. This work illustrated that shrimp-specific simplification could reduce computational cost, although its disease space and acquisition protocol differ from four-class Healthy/BG/WSSV/co-infection screening. In parallel, Querol et al. investigated MobileNetV3-Small and EfficientNetV2-B0 for WSSV monitoring, combined on-device considerations with saliency heatmaps, and explicitly discussed class imbalance [13]. The progression was therefore not merely from one architecture to another; it was a movement from offline classification toward resource-aware and interpretable screening.

2.3 Public Datasets and Data-Centric Development

The field expanded more rapidly after the release of reusable image resources. ShrimpDiseaseBD and its associated ShrimpDiseaseImageBD record introduced a four-class collection containing Healthy, BG, WSSV, and combined BG/WSSV images collected with farm collaboration and expert supervision [14], [15]. TigerShrimpBD provided an additional public source that was later used together with ShrimpDiseaseImageBD by a lightweight classification study [16], [17]. These releases made independent benchmarking more feasible, but image-level labels did not remove the need to audit repeated specimens, near duplicates, acquisition bias, or annotation uncertainty.

Adjacent aquatic-disease datasets are useful only when their domain difference is explicit. SalmonScan, for example, supports fish-disease image research but differs in species, symptom morphology, acquisition conditions, and class semantics [18]. Such datasets can demonstrate broader trends in aquatic computer vision, but they should not be treated as direct substitutes for shrimp-specific validation.

2.4 Lightweight, Explainable, and Deployable Systems

From 2024 onward, shrimp disease research increasingly combined classification with application-oriented requirements. SHRIMPAI implemented a mobile WSSV screening workflow [19], while Xu et al. and Fei et al. explored YOLO-based shrimp disease localization or detection [20], [21]. Ramachandran et al. combined an enhanced gated recurrent unit with wild-geese-migration optimization for WSSV-oriented shrimp classification [22]. These studies broadened the field from generic transfer learning toward disease-specific deployment and localization, although object detection remained different from instance segmentation because bounding boxes do not describe the exact disease-region boundary.

The 2025–2026 literature further diversified. Büyükarıkan evaluated multi-model ensembles; Aung et al. compared MobileNet and EfficientNet in a pathogen-oriented imaging setting; Raj et al. introduced a recurrent capsule architecture; and other studies combined classification with recommendations, IoT monitoring, or severity assessment [23–28]. ShrimpXNet is the closest classification reference for the present thesis because it combines background removal, transfer learning, MixUp, CutMix, adversarial training, and CAM-based explanation [29]. FeatherNetX represents a different direction: a compact architecture evaluated through five-fold cross-validation and two public shrimp image sources [17]. Recent MobileNetV3–KNN and explainable ensemble studies continue this trend toward compact or interpretable systems, while additional conference work has continued the direct shrimp-disease classification line [30–32].

3. Key Studies and Theories

3.1 Shrimp Disease Classification

Shrimp disease classification studies differ substantially in label space. Some use six disease categories, some focus only on WSSV, and others classify pathological tissue or a small set of visibly distinct syndromes. Duong-Trung et al. reported an early multi-disease transfer-learning result, whereas Wang et al. designed a compact LeNet derivative for three Penaeus vannamei diseases [11], [12]. WSSV-focused systems subsequently evaluated mobile backbones or application workflows [13], [19], [22]. These studies demonstrate feasibility, but their numerical results cannot be placed in a single ranking because the datasets and target definitions are not equivalent.

The more recent four-class literature is more directly relevant. ShrimpXNet evaluates several pretrained backbones on the public four-class dataset and incorporates regularization, adversarial training, and explanation methods [29]. FeatherNetX combines a compact backbone with public shrimp datasets and reports model size and computational complexity in addition to classification performance [17]. Büyükarıkan, Raj et al., and the explainable ensemble study demonstrate alternative ensemble, capsule, and multi-backbone directions [23], [25], [31]. Together, these papers show that the field has moved beyond asking whether CNNs can recognize shrimp disease; current questions concern efficiency, robustness, class imbalance, interpretability, external evaluation, and reproducibility.

ShrimpXNet must nevertheless be described carefully. It is a literature baseline and methodological reference for classification, not the architecture directly modified by the proposed classifier in this thesis. Its lack of a complete official reproduction package also means that any local reimplementation must document data protocol, preprocessing, augmentation, architecture, optimization, checkpoint selection, evaluation definitions, framework versions, and nondeterministic behavior rather than attributing differences only to a generic “different environment.”

3.2 Public Shrimp and Aquatic-Disease Datasets

Dataset papers make the experimental object explicit and therefore deserve separate analysis from model papers. ShrimpDiseaseBD documents the acquisition and class organization of the Bangladesh shrimp disease images, while the Mendeley record preserves the downloadable dataset version and DOI [14], [15]. TigerShrimpBD is relevant because it was used in later peer-reviewed lightweight classification work [16], [17]. The main limitation of these sources for instance segmentation is that they provide image-level disease categories rather than independently reviewed disease-region masks.

A dataset description is not sufficient evidence of split validity. Multiple views of the same physical specimen can produce highly similar images with consistent disease labels. If such views cross training and test partitions, the model may exploit specimen-specific appearance rather than disease-general evidence. Dataset comparison must therefore include label type, annotation provenance, source identity, repeated-subject risk, and whether the original publication reports a grouped or source-aware split.

3.3 Instance-Segmentation Foundations

Dense prediction developed through several architectural stages. Fully convolutional networks and U-Net established pixel-wise semantic prediction and encoder–decoder recovery of spatial detail [33], [34]. Mask R-CNN then extended two-stage detection with a parallel mask branch, providing a foundational framework for instance-level masks [35]. YOLACT separated prototype-mask generation from instance-specific coefficients, showing that real-time instance segmentation could be obtained with a one-stage formulation [36].

Later systems targeted boundary quality, ROI-free prediction, dynamic masks, and real-time query processing. PointRend treated segmentation as iterative rendering of uncertain points; CondInst generated instance-specific dynamic mask heads without ROI operations; SOLOv2 represented instances by location and dynamic kernels; and Mask2Former unified several segmentation tasks through masked attention [37–40]. YOLACTEdge, FastInst, and MobileInst moved the efficiency discussion closer to edge or mobile constraints [41–43]. These methods are not shrimp-specific, but they establish the design space for disease-region masks and clarify why classification accuracy alone cannot validate localization quality.

3.4 Lightweight Classification Architectures

The classification baseline families benchmarked by this project originate from a long sequence of architectural improvements. AlexNet, VGG, GoogLeNet, ResNet, DenseNet, and Xception established progressively deeper, multi-branch, residual, densely connected, and depthwise-separable representations [44–49]. SqueezeNet and MobileNet then made parameter count and deployment cost explicit design objectives [50], [51].

Mobile-oriented research subsequently introduced channel shuffling, inverted residuals, platform-aware search, hardware-aware activations, compound scaling, cheap feature generation, reparameterization, and hybrid CNN–transformer designs [52–65]. These papers provide the original architectural context for the TIMM screening table. Their relevance is not that every model is suitable for the final application, but that they allow the thesis to compare accuracy and Macro-F1 against parameter count, model size, latency, and conversion feasibility.

3.5 Imbalance-Aware Learning

Disease datasets are often imbalanced because Healthy images and common disease classes may be easier to collect than mixed or less frequent conditions. Focal Loss reduces the influence of easy examples; Class-Balanced Loss reweights classes using the effective number of samples; LDAM introduces class-dependent margins; and Asymmetric Loss applies different focusing behavior to positive and negative examples [66–69]. These are existing published methods and can be compared theoretically. The project-specific ASL–LDAM formulation, however, is not an existing related-work method and is therefore introduced only in the Methodology chapter.

The literature also warns against assuming that one imbalance method is universally superior. Loss behavior depends on whether the task is multiclass or multilabel, the severity of imbalance, label noise, calibration, sampling strategy, and model capacity. A fair thesis comparison therefore keeps the split, backbone, training schedule, augmentation, checkpoint selection, and metric definitions controlled when evaluating an imbalance-aware objective.

3.6 Attention and Feature Recalibration

Attention modules seek to redistribute representational capacity toward informative channels or spatial regions. Squeeze-and-Excitation models channel dependencies; CBAM applies sequential channel and spatial attention; ECA reduces channel-attention overhead; Coordinate Attention preserves positional information; SimAM derives parameter-free neuron importance; and Triplet Attention models cross-dimensional interactions [70–75]. These methods provide published foundations for controlled attention experiments in segmentation.

The review does not classify the project-specific attention configuration or DCFR module as an existing system. DCFR is a custom, independent feature-recalibration component, and its design and evidence belong to the proposed methodology. Likewise, an attention configuration selected through the project's instance segmentation experiments is a proposed configuration rather than a peer-reviewed baseline. Related work should explain the adopted foundations without assigning publication status to the project's own design.

3.7 Fourier-Domain Processing for Segmentation

Frequency-domain research provides a theoretical basis for examining image degradation and texture-sensitive segmentation. Yin et al. showed that robustness trade-offs can be interpreted through the frequency concentration of corruptions and augmentation effects [76]. Fourier Domain Adaptation transferred low-frequency amplitude information between source and target images for semantic segmentation, while a later Fourier framework investigated domain generalization through frequency manipulation [77], [78].

These studies motivate three segmentation-specific questions: whether Fourier transformations improve robustness under task-appropriate degradation; whether they preserve or enhance evidence for small disease regions; and whether they produce measurable benefits for instance segmentation. Exist-

ing literature does not answer these questions for BG/WSSV masks. The project's Fourier configurations and experimental answers are therefore evaluated later as proposed research, not listed here as published existing methods.

3.8 Robustness and Explainable AI

Robustness research distinguishes clean-test performance from stability under distribution shift. FGSM established a simple adversarial perturbation mechanism; mixup and CutMix regularized training through sample interpolation or region replacement; ImageNet-C formalized common-corruption benchmarking; and AugMix improved corruption robustness through diverse augmentation chains [79–83]. These methods support a controlled robustness protocol but do not justify claiming resilience to every farm condition.

Explainability research progressed from class activation mapping to gradient- and score-based variants. CAM links global-average-pooled features to class scores; Grad-CAM and Grad-CAM++ produce class-discriminative heatmaps; Score-CAM weights activation maps using forward scores; and Eigen-CAM produces a class-independent principal-component visualization [84–88]. Heatmaps can identify influential regions and expose obvious background dependence, but they do not establish causality, clinical validity, or pixel-accurate disease boundaries. This distinction is important when comparing XAI with instance segmentation.

3.9 Leakage-Aware and Reproducible Evaluation

Data leakage occurs when training or model-selection procedures gain information that would not be available in deployment. Kaufman et al. formalized leakage as an evaluation-design problem rather than merely a coding error [89]. In medical imaging, Zech et al. showed that site-specific confounding can weaken cross-domain generalization; Roberts et al. documented recurring methodological pitfalls; and Tampu et al. quantified inflated test accuracy caused by leakage [90–92].

The analogous shrimp-dataset risk is repeated-specimen leakage. If several photographs of the same shrimp appear in different partitions, an image-level random split can overestimate generalization. A specimen-grouped split reduces this specific risk by keeping all known views of one specimen together. It does not eliminate label error, acquisition-site bias, temporal confounding, or undocumented duplicates. Reproducible evaluation therefore requires split manifests, stable class ordering, identifiable checkpoints, recorded seeds, saved configurations, and traceable metric scripts.

3.10 Mobile and Edge Deployment

Mobile deployment requires more than a low parameter count. Model operators must be supported by the target runtime, preprocessing must match training, tensor order and class mapping must remain stable, and latency must be measured on representative hardware. MobileNet and later mobile architectures make hardware efficiency an explicit design objective [51], [53], [55], [65]. Shrimp-specific edge and mobile studies demonstrate the practical value of on-device or near-device screening [13], [19], [27].

The recent YOLO lineage also matters because unified implementations can support classification and instance segmentation within one tooling ecosystem. The original YOLO paper established single-stage real-time detection, YOLOv10 advanced end-to-end NMS-free detection, and the YOLO26 technical paper describes a unified family covering classification and instance segmentation [93–95]. Object-detection results are not treated as a research outcome of this thesis; the relevance is the shared deployment-oriented framework and the availability of lightweight classification and segmentation variants.

4. Technological Advancements

The technological development of shrimp-disease computer vision is best understood as a progression in both capability and evidential rigor. Early image-based studies asked whether visible disease patterns could be recognized at all. Later work introduced reusable deep representations, increasingly compact

model families, pixel- and instance-level localization, imbalance-aware learning, robustness analysis, explainable AI, leakage-aware validation, and finally deployment-oriented engineering. These developments are related, but they solve different problems. A compact classifier can reduce deployment cost without proving that the prediction is based on the disease region; a segmentation model can provide spatial evidence without solving data leakage; and a mobile demonstration can establish operational feasibility without establishing clinical validity.

For this reason, technological advancement in the present review is not treated as a simple sequence of newer architectures replacing older ones. The more meaningful progression is from a narrow prediction task toward a complete evidence chain: image acquisition, representation learning, efficient modeling, spatial localization, controlled evaluation, reproducible partitioning, and deployable inference. Each step introduces new benefits as well as new failure modes that must be considered separately. The following subsections therefore analyze how the field moved from manual visual judgment toward increasingly structured and traceable computer-vision systems, while keeping the biological and methodological boundaries explicit.

4.1 From Manual Inspection to Image-Based Screening

Conventional shrimp-health assessment begins with direct observation of external signs, behavior, water conditions, and production context, followed when necessary by microscopy, histopathology, molecular assays, or other laboratory procedures. The biological literature reviewed in Sections 1 and 2 explains why these procedures remain central: visible signs can overlap across pathogens, the same disease can appear differently across infection stages, and environmental stress can produce abnormalities that resemble infectious disease [1–9]. In other words, an external image is informative but incomplete. A photograph captures a visible phenotype at one moment; it does not directly establish pathogen identity, disease progression, or the causal mechanism behind the observed sign.

Image-based screening emerged because this limitation does not make visual evidence useless. Farmers and technicians already rely on visible appearance as an initial indication of abnormality. Computer vision changes that informal step by making it more repeatable and easier to record. Instead of depending only on memory or subjective comparison, a model can map an image to a defined label space, return confidence values, preserve the image for later review, and apply the same computational rule to every sample. Early transferred-CNN studies showed that multi-disease shrimp images could be classified automatically [11], while compact and mobile-oriented systems later demonstrated that the same screening idea could be implemented with lighter architectures or application workflows [12], [13], [19]. These studies established the feasibility of automated visual screening without removing the need for expert or laboratory confirmation.

The transition from manual inspection to image-based screening also changes the requirements placed on the data. A human observer can implicitly combine information from the pond, the animal, previous observations, and farm history. An image model normally receives a much narrower input. Consequently, acquisition conditions become part of the technical problem. Illumination, blur, viewing angle, image compression, background material, camera processing, and the proportion of the shrimp visible in the frame can all affect the pixels presented to the model. A laboratory or curated dataset can reduce some of this variation, but a mobile screening system ultimately has to confront it. This is why later work increasingly considers robustness, preprocessing, and deployment rather than clean classification accuracy alone [13], [19], [29].

The progression also exposes an important distinction between *screening consistency* and *diagnostic certainty*. A model may be useful if it consistently flags visually suspicious shrimp for follow-up, even when it is not qualified to diagnose disease. Conversely, a high classification score on a curated dataset should not be interpreted as evidence that the system can replace veterinary examination. The correct technological role is therefore a decision-support layer: rapid, non-invasive, repeatable, and potentially deployable near the point of image capture, but bounded by the quality of the image labels and by the absence of direct pathogen confirmation in ordinary photographs.

This distinction influences system design. A screening model should make its output understandable enough to support follow-up rather than obscure it. Image-level labels are useful for speed, but localization or explanation can provide additional evidence about where the model is responding. Likewise, preserving the original image, prediction, confidence, and processing history can be more valuable op-

erationally than returning a single untraceable class label. The literature's movement toward mobile applications, saliency visualization, and localization therefore represents a broader change in purpose: from proving that a CNN can recognize a disease category toward building a screening process whose outputs can be inspected, stored, and interpreted within an explicit diagnostic boundary.

4.2 From Handcrafted Features to Transfer Learning

Before deep representation learning became dominant, image-analysis systems generally depended on manually designed descriptors. Researchers chose which color statistics, textures, edges, shapes, or local patterns should represent the image and then supplied those features to a conventional classifier. This approach can work well when the visual cue is stable and known in advance, but it places a large part of the modeling decision before the data have been fully explored. Disease appearance is especially awkward for such a pipeline because relevant cues may be small, irregular, distributed across the body, or entangled with illumination and background. A handcrafted representation that is effective for one acquisition setting can therefore become fragile when the appearance changes.

Transfer learning changed the practical starting point. Instead of designing the feature space from scratch, the model reuses visual hierarchies learned on a much larger source dataset and adapts them to the target shrimp images. This is particularly attractive for shrimp-disease research because public domain datasets are modest compared with general-purpose image collections. Duong-Trung et al. demonstrated that transferred CNNs could support multi-disease shrimp classification [11]. Subsequent shrimp-specific work explored compact custom CNNs, MobileNet- and EfficientNet-oriented models, ensembles, recurrent or capsule-based designs, and combinations of preprocessing and transfer learning [12], [13], [23–25], [29], [31]. The technological change is therefore not just that neural networks became deeper. Representation learning became reusable, reducing the need to encode every discriminative visual pattern manually.

Transfer learning also made architecture selection an empirical question. Once several pretrained families are available, a researcher can compare how different feature extractors behave under the same target task rather than committing to one representation a priori. This is useful because architecture families encode different assumptions about channel mixing, receptive field, depth, normalization, parameter sharing, and computational efficiency. A model that is strong on a generic benchmark does not automatically dominate on shrimp disease images, especially when the target dataset is small, imbalanced, or visually homogeneous. Direct screening across multiple candidate families therefore provides stronger evidence than selecting a backbone only because it is popular or nominally lightweight.

However, the flexibility of transfer learning introduces additional confounding factors. Two reported backbones are only meaningfully comparable when preprocessing, image resolution, split construction, augmentation, optimization, checkpoint selection, and metric calculation are controlled. A pretrained network can appear superior because it received a more favorable crop, stronger augmentation, a different class distribution, or a test split containing near-duplicate views. The literature reviewed in this chapter varies substantially in these details, which is why headline percentages are not treated as a common leaderboard. Transfer learning expands the design space, but it also increases the need for a disciplined evaluation protocol.

Another important consequence is that the term "transfer learning" covers several levels of adaptation. A study may freeze most of the backbone and train only a classifier head, fine-tune the complete model, change the input preprocessing, or add task-specific modules on top of the pretrained representation. These choices affect both optimization and reproducibility. When a paper does not report the exact fine-tuning strategy, an independent implementation can legitimately produce a different result even if the named backbone is the same. Reproducible research therefore requires the transferred model to be specified together with initialization, preprocessing, trainable components, and the model-selection rule rather than by architecture name alone.

In shrimp-disease vision, this progression supports a more careful interpretation of prior work. The strongest contribution of transfer learning is not that it eliminates domain-specific design, but that it provides a stable starting representation from which domain-specific hypotheses can be tested. It allows the research question to move from "which handcrafted feature should represent disease?" toward "which pretrained representation and adaptation strategy provide the best evidence under a controlled

shrimp-specific protocol?" That shift is central to later developments in lightweight model selection, imbalance-aware optimization, attention, robustness, and deployment.

4.3 From Heavy CNNs to Lightweight Architectures

The early success of deep convolutional networks encouraged progressively larger models, but mobile and edge deployment exposed a different optimization objective. A server-side benchmark can tolerate substantial computation if it improves predictive quality. A resource-constrained application must also consider memory, storage, operator support, latency, and energy use. The literature therefore moved from treating efficiency as an afterthought toward designing it directly into the architecture. SqueezeNet reduced parameters aggressively; MobileNet popularized depthwise separable convolution; ShuffleNet emphasized efficient channel interaction; later families introduced inverted residuals, platform-aware architecture search, compound scaling, cheap feature generation, structural reparameterization, and lightweight CNN–transformer hybrids [50–65]. Although these families differ technically, they share the principle that efficiency should be designed rather than obtained only by shrinking a conventional network.

This change is important for shrimp screening because predictive quality and deployability do not necessarily move together. A larger model can improve feature capacity yet be too expensive for the target device. A very small model can reduce memory and latency yet lose class-balanced performance. Parameter count provides one view of complexity, but it is not sufficient by itself. Two networks with similar parameter counts may have different operation counts, activation-memory requirements, kernel implementations, or export compatibility. Serialized model size can differ from the raw parameter count because of numerical precision and graph representation. Measured latency can differ again because runtime libraries optimize some operators more effectively than others. For deployment-oriented comparison, architecture size, artifact size, and measured execution should therefore be treated as related but distinct forms of evidence.

The literature also shows why a "mobile" label should not be interpreted as a guarantee of real-device speed. Architecture papers often report theoretical FLOPs or measurements on a particular platform. A downstream application may use a different framework, precision, input resolution, thread count, or hardware backend. The same network can therefore behave differently after conversion. This is one reason why mobile-oriented shrimp studies such as Querol et al. and SHRIMPAI are useful complements to generic architecture papers [13], [19]: they move the discussion from theoretical efficiency toward application context, even though their disease scope differs from the four-class and instance-segmentation setting considered here.

A second consequence is that model selection becomes a multi-objective decision rather than a single-metric race. If two candidates have similar Accuracy but one has substantially lower Macro-F1, the smaller candidate may not actually be preferable for an imbalanced disease task. If a larger diagnostic model produces only a negligible predictive gain, its additional capacity may not be justified. Conversely, a compact model that is slightly slower under a particular software runtime may still be attractive if it exports reliably and preserves predictive quality. The relevant comparison is therefore a performance–efficiency frontier: predictive metrics are examined together with parameters, size, runtime evidence, and deployment constraints.

Larger networks still retain scientific value in this setting. They can serve as capacity controls that test whether a lightweight model is underfitting the problem. If a much larger candidate does not improve validation or class-balanced test performance under the same protocol, the result suggests that additional capacity alone is not the limiting factor. Such a model can therefore be useful even when it is excluded from final deployment. This distinction prevents two common mistakes: assuming that the largest available model is automatically the strongest scientific baseline, or assuming that the smallest model is automatically the best engineering choice.

The broader technological advancement is thus a change in what "better" means. For a deployment-oriented screening system, a defensible architecture is not simply the network with the highest isolated score. It is a model whose predictive evidence remains strong enough while its complexity, export path, and runtime behavior remain compatible with the intended device class. This reasoning justifies comparing multiple efficient families under a common protocol and retaining larger models as diagnostic references rather than treating architecture selection as a matter of brand or parameter count alone.

4.4 From Image-Level Labels to Instance-Level Evidence

Image classification, object detection, semantic segmentation, and instance segmentation provide progressively different forms of spatial evidence. A classifier answers which class best describes the image as a whole. Object detection adds a bounding box around a suspected region, but the box necessarily includes background pixels and does not specify the visible lesion boundary. Semantic segmentation predicts a class for each pixel, but pixels of the same class are ordinarily merged into a class-level map. Instance segmentation adds a further distinction by representing separate occurrences individually. These tasks should not be treated as interchangeable because they answer different questions and use different evaluation criteria.

The technical progression is visible in the general vision literature. FCN and U-Net established practical dense semantic prediction through encoder–decoder architectures [33], [34]. Mask R-CNN extended object detection with an explicit per-instance mask branch [35]. YOLACT demonstrated that prototype masks and instance coefficients could make instance segmentation substantially faster [36]. PointRend focused on uncertain boundary points, while CondInst and SOLOv2 explored dynamic or location-based instance-mask prediction without the same ROI formulation [37–39]. Mask2Former introduced a more unified query-based segmentation view, and later systems such as YOLACTEdge, FastInst, and MobileInst pursued faster or mobile-oriented dense prediction [40–43]. The direction of travel is clear: the field moved from whole-image recognition toward increasingly explicit spatial evidence while simultaneously trying to preserve computational efficiency.

Shrimp-specific research has not progressed through exactly the same stages. Recent YOLO-based studies provide disease localization through bounding boxes [20], [21], which is useful because it demonstrates that visible abnormalities can be localized rather than only classified. However, a box cannot express the irregular shape of a black-gill region, a cluster of visible white spots, or multiple separated regions within the same image. For research questions involving lesion shape, boundary quality, or multiple disease regions, mask-level evaluation is therefore stricter than box-level detection. The general instance-segmentation literature supplies the architectural foundations, while shrimp-specific mask evidence remains comparatively limited.

Moving to masks also shifts part of the scientific burden from architecture to annotation. Image-level labels require deciding what class the photograph represents. Instance masks require deciding where the visible evidence begins and ends, whether disconnected regions belong to one or several instances, how co-infection is represented, and how ambiguous boundaries are handled. Healthy images create an additional requirement: they should produce an empty disease-mask target rather than being ignored, otherwise false-positive segmentation on healthy shrimp can remain hidden. These choices are not implementation trivia. They define the target that the model is being trained and evaluated to reproduce.

The distinction between semantic and instance segmentation is especially important when interpreting U-Net-style baselines. A standard U-Net predicts semantic class masks [34]. It does not natively separate multiple instances of the same class. Dice, IoU, or mIoU from a semantic protocol therefore cannot be placed directly beside instance-level mask AP or mAP and interpreted as a common ranking unless a matched evaluation design and, where required, an instance-extraction procedure are defined. The issue is not that U-Net cannot be trained on the same images; it is that the endpoint and metric must be aligned before a numerical comparison is scientifically meaningful.

Mask-level evidence should also not be confused with clinical ground truth. A technically consistent polygon annotation can support computer-vision experiments, but its validity depends on the annotator's ability to identify visible disease evidence. Public shrimp datasets are primarily organized around image-level categories and do not provide independently reviewed disease-region masks [14–16]. A project that constructs masks from those images can evaluate its own annotation protocol, but it must state whether annotations are single-annotator, whether expert review exists, and whether inter-annotator agreement has been measured. Architectural sophistication cannot compensate for uncertainty in the target labels.

The technological advancement from classification to instance segmentation is therefore both a modeling and evidence transition. It enables the system to answer not only "what class was predicted?" but also "where is the visible evidence located?" At the same time, it introduces stricter requirements for annotation quality, healthy-negative handling, metric selection, and interpretation. This is why image-level classification, explanation heatmaps, boxes, semantic masks, and instance masks should be treated

as complementary evidence types rather than progressively interchangeable versions of the same result.

4.5 From Clean Accuracy to Robust and Transparent Evaluation

As computer-vision models became more capable, evaluation expanded beyond one clean-test Accuracy value. This change was necessary because a single aggregate score can hide several distinct failure modes. In an imbalanced dataset, the majority class can dominate Accuracy even when a minority disease class performs poorly. A model can achieve a strong score on a curated test set yet degrade sharply under blur, low light, contrast change, noise, or compression. Two models with similar Accuracy can also differ in their classwise precision–recall balance. The technological progression in evaluation therefore involves both additional metrics and additional test conditions.

Imbalance-aware learning addresses one part of this problem. Focal Loss reduces the influence of easy examples, class-balanced weighting adjusts the contribution of classes according to sample frequency, LDAM introduces class-dependent margins, and ASL uses asymmetric focusing behavior [66–69]. These methods do not guarantee improvement on every dataset, but they reflect a broader shift away from treating every classification error as equally informative. For shrimp disease images, where class counts and visual difficulty can differ, class-balanced metrics such as Macro-F1 are especially useful because each class contributes equally to the final average. Accuracy remains informative, but it should be interpreted together with Macro-F1 and, where available, classwise precision, recall, or confusion evidence.

Data augmentation and robustness evaluation extend the question from class balance to input variation. mixup and CutMix alter the training distribution through sample combination, while AugMix and common-corruption benchmarks provide tools for evaluating or improving stability under controlled appearance changes [80–83]. ShrimpXNet further demonstrates the use of augmentation, adversarial training, and CAM-based interpretation in a four-class shrimp setting [29]. The important methodological point is that augmentation and robustness testing serve different roles. Augmentation changes what the model sees during training; a corruption benchmark measures how the trained model behaves when a defined perturbation is applied at evaluation time. Reporting that a model was augmented is therefore not equivalent to demonstrating robustness.

Controlled corruptions are useful precisely because their severity and type can be specified. Noise, defocus blur, contrast reduction, and low-light transformations can approximate some acquisition problems that a mobile camera may encounter. However, they remain synthetic. Passing a finite corruption suite does not establish robustness to every pond, background, camera pipeline, or disease presentation. Corruption testing should therefore be interpreted as stress evidence under known transformations, not as a universal field-generalization claim. The same caution applies to adversarial tests: they reveal sensitivity under a defined perturbation model rather than predicting ordinary farm behavior.

Explainable AI adds a different form of transparency. CAM, Grad-CAM, Grad-CAM++, Score-CAM, and Eigen-CAM visualize class-associated activation or attribution patterns in different ways [84–88]. These maps can reveal whether a classifier is concentrating on the shrimp body, an apparent disease region, or an obvious background cue. They are valuable for qualitative error analysis and for detecting implausible attention patterns. They are not, however, segmentation ground truth. A saliency map can be diffuse, resolution-limited, or sensitive to model internals, and visual plausibility does not prove causal reasoning. For this reason, XAI should complement rather than replace quantitative classification and localization evidence.

Transparent evaluation also requires keeping evidence from different regimes separate. Clean-test performance, corruption performance, adversarial behavior, explanation quality, and instance-segmentation accuracy answer different questions. Combining them into one unsupported claim of “robustness” would erase the conditions under which each result was obtained. A stronger reporting practice states the dataset, split, transformation, metric, and model checkpoint for every claim and avoids reusing one result as evidence for a different property.

The field’s progress is therefore methodological as much as architectural. Modern evaluation asks not only whether a model is accurate, but whether its performance is balanced across classes, whether it remains stable under controlled input changes, whether its explanations are at least qualitatively plausible, and whether spatial localization supports the image-level decision. No single test is sufficient.

The most defensible interpretation comes from multiple complementary evidence streams whose limitations remain explicit.

4.6 From Random Splits to Leakage-Aware Protocols

A model can only demonstrate generalization when the test set represents information that was not effectively available during training. Random image-level splitting is convenient because it preserves approximate class proportions with little manual effort. It becomes problematic, however, when images are clustered by specimen, patient, farm, source, acquisition session, or near-duplicate capture. The broader leakage literature shows that subject overlap, site confounding, preprocessing leakage, and other hidden dependencies can produce optimistic test results even when the training loop itself is implemented correctly [89–92]. In such cases, the numerical score is real for the chosen split but answers a weaker question than the researcher may intend.

Shrimp imagery creates a particularly clear example. A physical shrimp can be photographed multiple times from related viewpoints. Those photographs may share body shape, pigmentation, damage patterns, background, lighting, and camera characteristics. If one view enters the training set and another enters the test set, the model may exploit specimen-specific appearance rather than learning disease features that transfer to a new shrimp. The images are not byte-for-byte duplicates, so ordinary duplicate detection may not flag the problem. Nevertheless, the statistical unit of independence is closer to the specimen than to the photograph.

Grouped-specimen splitting addresses this issue by assigning all recognized views of a specimen to one partition. The objective is not to make the task artificially harder, but to align the partitioning unit with the desired generalization claim. If the intended deployment scenario is a previously unseen shrimp, the evaluation should avoid showing the model another view of the same animal during training. This principle is analogous to patient-level splitting in medical imaging or subject-level splitting in biometric data. The same reasoning applies when integrating multiple datasets: if source identity is known, partitioning should be performed in a way that prevents later merging from moving related records across train, validation, and test sets.

Leakage-aware evaluation requires more than setting a random seed. A fixed seed makes a split reproducible, but it does not make the split scientifically valid. Reproducibility and validity are separate properties. A perfectly reproducible image-level random split can still contain specimen overlap. Conversely, a grouped split can still be weakened by label error, source confounding, undocumented duplicates, or an unrepresentative test distribution. The purpose of leakage control is therefore narrower and more defensible: remove a known pathway through which test information could become too similar to training information.

Metadata quality limits how far grouping can be enforced. Public datasets do not always provide a reliable specimen identifier. Filenames may encode some identity information for part of the collection but not for every image. When identity cannot be reconstructed, forcing an invented group assignment would create false certainty. A rigorous protocol should therefore distinguish recognized groups from unmatched records and document how each subset is handled. This is preferable to silently assuming that every filename represents an independent animal.

Leakage analysis can also explain why a more careful protocol sometimes produces lower performance. A decrease after grouping does not necessarily mean the model has become worse. It may indicate that the earlier split contained easier, highly related test samples. In that situation, the lower grouped score can be a more credible estimate of generalization. This is a crucial conceptual shift: evaluation quality is not measured by how high the final metric is, but by how well the experiment supports the claim being made.

The technological progression from random splitting to leakage-aware protocols is therefore part of model development, not a bookkeeping detail. Dataset partitions define the experiment just as strongly as the architecture and loss function. For shrimp disease research, explicit specimen grouping, source-aware integration, fixed split manifests, and documented unmatched cases provide a more traceable basis for comparison than an undocumented image-level random split. They do not remove every threat to validity, but they make one of the most consequential ones visible and controllable.

4.7 From Offline Models to Integrated Mobile Systems

The final technological shift connects experimental models to a complete inference workflow. In an offline study, success can be demonstrated by loading a checkpoint, running inference in the research environment, and reporting evaluation metrics. A mobile system must preserve the same model semantics after export, conversion, integration, and interaction with the application. Shrimp-specific work such as Querol et al., SHRIMPAI, and IoT-oriented systems shows that disease screening can move beyond notebooks into edge or application contexts [13], [19], [27]. This move creates a new set of engineering requirements that are largely invisible in a conventional model benchmark.

The first requirement is an explicit input contract. Image resolution, color order, normalization, cropping, background processing, and tensor data type must match the assumptions used during model evaluation. A model can be exported successfully yet produce inconsistent predictions if the Android application constructs the input tensor differently from the training pipeline. The same is true of the output contract: class order, confidence interpretation, mask dimensions, thresholding, and postprocessing must remain synchronized between the model artifact and the application code. Deployment correctness therefore depends on preserving semantics, not merely on producing a file that the runtime can open.

The second requirement is export and runtime compatibility. A research model may contain operators that are convenient in PyTorch but unsupported or inefficient in a mobile runtime. Conversion to TensorFlow Lite or another deployment format can change precision, graph structure, or available acceleration paths. FP32 and FP16 artifacts, for example, can differ in size and runtime behavior even when they represent the same conceptual network. The target device, thread count, CPU delegate, GPU delegate, NNAPI path, and operating-system version can all influence measured execution. This is why parameter count, serialized artifact size, and real-device runtime should be reported separately rather than treated as interchangeable measures of efficiency.

A combined classification-and-segmentation application introduces an additional systems question: when should each model execute? Running both models for every image simplifies control flow but wastes compute on cases that do not need localization. A conditional pipeline can run classification first and invoke segmentation only when a disease class exceeds the configured threshold. Such a design can reduce unnecessary work, but it means that application latency becomes branch-dependent. A classification-only case and a segmentation-triggered case are not measuring the same execution path. Average application time must therefore be interpreted together with branch composition and workload rather than as pure neural-network latency.

The third requirement is measurement provenance. Mobile elapsed time can include image decoding, resizing, tensor creation, interpreter execution, mask reconstruction, result formatting, storage, and interface overhead. Notebook or GPU timing may isolate a different scope. Comparing those values directly can therefore be misleading. A scientifically useful deployment study records what the timer includes, the warm-up policy if applicable, the tested hardware, the number of runs, and whether the same images were used across devices. Without those details, a fast number can be visually impressive while remaining difficult to interpret.

Real-world image acquisition adds another boundary. A mobile application can be exercised with photographs captured outside the curated dataset to test whether the software path accepts realistic inputs, whether preprocessing remains stable, and whether the complete pipeline returns outputs under varied lighting, background, viewpoint, or blur. Unless those photographs have independent veterinary or laboratory ground truth, however, they cannot establish field diagnostic accuracy. Operational validation and diagnostic validation are different experiments. The former demonstrates that the system runs under realistic acquisition conditions; the latter requires trustworthy disease labels and an evaluation design capable of measuring correctness.

Integration also extends beyond inference. A usable application may include image capture or gallery selection, result display, history management, authentication, cloud synchronization, or administrator services. These components matter because they determine how the model output is stored and interpreted by the user, but they should not be confused with model performance. A Firebase-backed history function, for example, can improve traceability without improving classification Accuracy. Keeping software functionality and predictive evidence separate prevents application completeness from being mistaken for scientific validation.

The progression from offline models to integrated mobile systems is therefore not simply a change of hardware. It is a transition from a model artifact to a traceable software contract. The model, preprocessing, class semantics, dispatch rules, exported precision, runtime path, device configuration, and user-facing interpretation all have to remain consistent. A successful mobile research prototype is one that can reproduce the intended inference behavior on the target devices and can document what was measured. It remains a preliminary screening system unless clinical ground truth, farmer-facing validation, and broader hardware studies provide evidence for stronger claims.

5. Comparison of Existing Systems

The following tables organize the literature by chronology, task, dataset, evaluation coverage, and limitation. Numerical results are preserved only when the original source reports them clearly. They are not treated as directly comparable rankings because the studies differ in species, disease definitions, image modality, dataset size, split policy, augmentation, architecture, and evaluation metric.

Table 3.1: Chronological evolution of related work

Year	Study / method	Task and main contribution	Relevance to this thesis
1997	Flegel [1]	Shrimp disease review: Summarized major viral diseases affecting black tiger shrimp.	Establishes the biological and economic context.
1998	Lightner [2]	Disease control: Reviewed practical viral-disease control strategies.	Defines conventional management boundaries.
2009	Walker and Mohan [3]	Disease emergence: Connected viral emergence, impact, and health management.	Motivates early screening and surveillance.
2011	Lightner [4]	Shrimp virology review: Reviewed farmed-shrimp virus diseases in the Americas.	Shows geographic and pathogen diversity.
2012	Stentiford et al. [5]	Food-supply risk: Linked crustacean disease to future seafood supply.	Supports project significance.
2012	Flegel; Seibert and Pinto [6], [7]	Pathogen history: Reviewed major shrimp pathogens and viral dispersion.	Explains why visible symptoms require cautious interpretation.
2016	Thitamadee et al. [8]	Disease-threat review: Synthesized current threats for cultivated penaeid shrimp.	Defines realistic disease complexity.
2018	Shinn et al. [9]	Economic impact: Assessed disease-related production costs in Asian shrimp farming.	Strengthens practical motivation.
2015–2017	FCN, U-Net, Mask R-CNN [33–35]	Segmentation foundations: Advanced pixel-wise and instance-level mask prediction.	Provides mask-localization foundations.
2016–2019	YOLO, YOLACT [36], [93]	Real-time vision: Moved detection and instance segmentation toward real-time inference.	Supports lightweight localization context.

Year	Study / method	Task and main contribution	Relevance to this thesis
2017–2024	Mobile architecture families [51–53], [55], [65]	Mobile classification: Designed efficient operators and hardware-aware models.	Supports edge-oriented model selection.
2017–2021	Focal, class-balanced, LDAM, ASL [66–69]	Imbalanced learning: Introduced focusing, reweighting, margin, and asymmetric objectives.	Provides imbalance-learning foundations.
2016–2020	CAM families [84–88]	Explainable AI: Produced visual evidence from classifier activations.	Supports qualitative interpretation.
2018–2020	mixup, CutMix, ImageNet-C, AugMix [80–83]	Robustness: Expanded regularization and common-corruption evaluation.	Supports robustness protocol design.
2019–2021	Fourier robustness and adaptation [76–78]	Frequency-domain learning: Connected frequency content with robustness and domain shift.	Motivates Fourier segmentation experiments.
2020	Duong-Trung et al. [11]	Shrimp classification: Applied transferred CNNs to multi-disease shrimp images.	Early direct classification baseline.
2020–2024	CondInst, Mask2Former, MobileInst [38], [40], [43]	Efficient instance segmentation: Improved dynamic, query-based, and mobile-oriented masks.	Defines modern segmentation design space.
2023	Wang et al. [12]	Compact shrimp classification: Modified LeNet for rapid Penaeus vannamei disease recognition.	Shows shrimp-specific lightweight design.
2023	Querol et al. [13]	Edge WSSV recognition: Compared mobile backbones, imbalance handling, and saliency.	Closest early edge/XAI direction.
2024	SHRIMPAI [19]	Mobile WSSV application: Integrated CNN screening into a mobile application.	Provides mobile workflow precedent.
2024	Xu et al. [20]	Shrimp disease detection: Combined improved YOLOv8 with multiple visual features.	Supports localization context, not mask evidence.
2024	Ramachandran et al. [22]	WSSV classification: Combined an enhanced GRU with wild-geese-migration optimization.	Adds disease-specific sequence-modeling evidence.
2025	ShrimpDiseaseBD/ ImageBD [14], [15]	Public dataset: Released a documented four-class shrimp-disease image source.	Primary classification dataset reference.
2025	TigerShrimpBD [16]	Public dataset: Released an additional tiger-shrimp image source.	Supports later multi-source evaluation.
2025	Büyükarıkan [23]	CNN ensemble: Evaluated multi-model ensemble strategies.	Shows ensemble performance and complexity trade-offs.

Year	Study / method	Task and main contribution	Relevance to this thesis
2025	Aung et al. [24]	Pathogen-image classification: Compared MobileNet and EfficientNet in a different imaging setting.	Adjacent evidence; not a direct smartphone baseline.
2025	Raj et al. [25]	Capsule/recurrent classification: Combined capsule, recurrent, attention, and optimization components.	Alternative architecture with high complexity.
2025	Fei et al. [21]	Lightweight detection: Reduced YOLOv8n complexity for shrimp disease detection.	Supports efficient localization context.
2026	ShrimpXNet [29]	Robust/XAI classification: Combined preprocessing, augmentation, adversarial training, and CAM.	Closest literature classification baseline.
2026	FeatherNetX [17]	Lightweight classification: Reported a compact model with cross-validation and two data sources.	Strong efficiency and reproducibility reference.
2026	MobileNetV3–KNN [30]	Early detection: Integrated a mobile CNN with KNN classification.	Alternative compact deployment approach.
2026	Explainable ensemble CNN [31]	Explainable classification: Combined multiple backbones, imbalance handling, and Grad-CAM.	Recent direct XAI-oriented reference.
2026	IoT–ResNet monitoring [27]	Integrated monitoring: Combined image classification with pond sensing and edge inference.	Shows system-level deployment integration.
2026	YOLO26 [95]	Unified vision family: Provides classification and instance segmentation variants in one pipeline.	Relevant engineering backbone for classification and instance segmentation variants.

Table 3.1 shows a continuous change in research emphasis. The earliest literature defined the biological problem and the limits of conventional disease management. General computer vision then provided classification, segmentation, robustness, explainability, and mobile-efficiency foundations. Shrimp-specific studies first adopted transfer learning, subsequently introduced compact architectures and mobile workflows, and only recently began combining public datasets, robustness, XAI, external sources, or integrated sensing. This chronology also exposes an imbalance in the literature: classification systems are increasingly mature, whereas peer-reviewed shrimp disease instance segmentation and leakage-aware specimen splitting remain scarce.

Table 3.2: Representative shrimp disease classification systems

Study	Dataset and method	Reported evidence and strength	Limitation for this project
Duong-Trung et al. [11]	Study-specific six-disease image set; Transferred CNN classification	90.02% accuracy reported; Early direct field-image study	No masks, mobile validation, or grouped split reported
Wang et al. [12]	Three Penaeus vannamei disease classes; Improved LeNet	Rapid compact recognition reported; Shrimp-specific lightweight design	Different classes and protocol from this thesis
Querol et al. [13]	WSSV monitoring dataset; MobileNetV3-Small / EfficientNetV2-B0	F1 = 0.72 / 0.99; Edge focus, imbalance discussion, saliency	Binary task and no mask evaluation
SHRIMPAI [19]	WSSV mobile screening; Sequential CNN and mobile app	Successful tests on two shrimp species reported; Complete application-oriented workflow	Limited sample scale and binary disease focus
Ramachandran et al. [22]	WSSV-oriented shrimp images; Enhanced GRU with wild-geese optimization	Improved WSSV-classification performance reported; Disease-specific optimized recurrent model	Single-disease setting
Büyükarıkan [23]	ShrimpDiseaseBD; Multi-model CNN ensemble	Study hypothesis and results exceed 0.90 accuracy; Strong ensemble comparison	Higher complexity; no mask localization
Aung et al. [24]	Pathogen/histopathology images; MobileNet and EfficientNet	Accuracy above 95% reported; Lightweight pathology recognition	Different modality from farm smartphone images
Raj et al. [25]	Shrimp disease images; Recurrent capsule and hybrid optimization	Precision about 94.9% reported; Rich spatial/channel modeling	Complex pipeline and limited deployment evidence
ShrimpXNet [29]	ShrimpDiseaseImageBD; four classes; Transfer learning, preprocessing, MixUp/CutMix, FGSM, CAM	ConvNeXt-Tiny 96.88% test accuracy reported; Closest robustness/XAI classification reference	Reproduction code is not fully specified
FeatherNetX [17]	ShrimpDiseaseImageBD + TigerShrimpBD; Compact custom classifier; five-fold CV	93% average accuracy; 0.739M parameters; Strong efficiency reporting and multi-source use	Desktop deployment; no instance masks
Explainable ensemble [31]	Enhanced aquaculture image data; DenseNet121 + ConvNeXt-Tiny + SE-ResNeXt50	96.43% precision reported; Imbalance handling and Grad-CAM	Ensemble complexity and short conference report
IoT–ResNet system [27]	Four-class image set + pond sensors; Transfer-learned ResNet-50 and IoT monitoring	Accuracy 0.8559; F1 0.8552; edge latency reported; End-to-end monitoring context	No instance segmentation or leakage audit

Table 3.2 demonstrates why reported percentages must not be compared as if they came from a common benchmark. The studies vary from binary WSSV recognition to four- or six-class disease classification, and some use tissue or histopathology images rather than whole-shrimp farm photographs. ShrimpXNet is the closest literature reference because its public dataset, preprocessing, augmentation, adversarial training, and XAI overlap with the classification concerns of this thesis. FeatherNetX contributes stronger efficiency and multi-source evidence. Neither paper provides the mask-level BG/WSSV evaluation or specimen-grouped protocol required by the instance segmentation track.

Table 3.3: Localization, instance segmentation, and evaluation methods

Study / method	Task and main idea	Strength	Limitation
FCN / U-Net [33], [34]	Semantic segmentation; Encoder–decoder pixel prediction	Strong dense-prediction foundation	Does not inherently separate multiple instances
Mask R-CNN [35]	Instance segmentation; Two-stage detection with parallel mask branch	Accurate, established baseline	Higher computational and proposal-stage overhead
YOLACT [36]	Real-time instance segmentation; Prototype masks plus instance coefficients	Good speed–accuracy design	Prototype quality can limit fine boundaries
PointRend [37]	Boundary refinement; Iteratively predicts uncertain points	Improves high-resolution boundaries	Adds refinement complexity
CondInst [38]	ROI-free instance segmentation; Dynamic instance-specific mask heads	High-resolution masks without ROI resizing	Detection-conditioned training remains complex
SOLOv2 [39]	One-stage instance segmentation; Location-based categories and dynamic kernels	Fast end-to-end formulation	Dense scene assumptions differ from disease regions
YOLACTEdge [41]	Edge instance segmentation; Edge-oriented YOLACT optimization	Direct edge deployment evidence	Video/edge design is not shrimp-specific
Mask2Former [40]	Universal segmentation; Masked-attention queries	Strong unified segmentation framework	Transformer complexity can conflict with mobile limits
FastInst / MobileInst [42], [43]	Real-time/mobile masks; Efficient query and mobile video designs	Demonstrate mobile-oriented dense prediction	Not validated for shrimp disease masks
Improved YOLO shrimp studies [20], [21]	Disease detection; Bounding-box localization with lightweight YOLO variants	Shrimp-specific localization evidence	Boxes do not provide disease-region boundaries
Fourier segmentation foundations [77], [78]	Domain adaptation/generalization; Frequency-amplitude manipulation	Motivates robustness and appearance studies	No direct BG/WSSV instance evidence

Study / method	Task and main idea	Strength	Limitation
Leakage literature [89–92]	Evaluation validity; Detects leakage, confounding, subject overlap, and methodological pitfalls	Protects generalization estimates	Requires specimen/source metadata that datasets may not provide

The localization literature develops from generic pixel prediction to explicit instance separation and then toward real-time or mobile operation. Shrimp-specific work is currently concentrated on bounding-box detection, while mask-level disease evidence depends mainly on general instance segmentation foundations. This gap justifies constructing BG/WSSV region masks, but it also imposes a quality limitation: a single-annotator mask set without independent expert review cannot be treated as a clinical ground truth.

Table 3.4: Public datasets relevant to shrimp and aquatic-disease vision

Dataset / source	Year and domain	Scale and labels	Relevance and limitation
ShrimpDisease ImageBD [14], [15]	2025; Shrimp disease	1,149 original images; four classes; Image-level disease labels	Primary four-class classification source; no expert-reviewed instance masks
TigerShrimpBD [16]	2025; Tiger shrimp images	Public Mendeley dataset; Image-level categories	Used by FeatherNetX; supports multi-source classification analysis
SalmonScan [18]	2024; Salmon disease	Public Data in Brief resource; Image-level disease labels	Adjacent aquatic-disease evidence; species/domain differ
Study-specific WSSV sets [13], [19]	2023–2024; WSSV monitoring	Collected for edge/mobile studies; Binary image labels	Useful deployment precedent; limited reuse and disease diversity
Pathogen-image dataset [24]	2025; Shrimp pathogens	Curated pathology-oriented images; Pathogen/tissue classes	Adjacent modality; not equivalent to whole-shrimp farm photographs

Table 3.4 separates reusable data resources from study-specific collections. ShrimpDiseaseImageBD is the most direct public classification source, while TigerShrimpBD supports additional source diversity. The absence of public, expert-reviewed BG/WSSV instance masks explains why the segmentation dataset had to be created by the project. It also means that segmentation results should be interpreted as evidence on a project-specific annotation protocol, not as validation against a recognized clinical mask benchmark.

Table 3.5: Research coverage of representative shrimp vision systems

Study	Core task coverage	Robustness and XAI	Deployment and evaluation validity
Duong-Trung et al. [11]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: No; leakage control: No; external evaluation: No
Wang et al. [12]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: No; leakage control: No; external evaluation: No

Study	Core task coverage	Robustness and XAI	Deployment and evaluation validity
Querol et al. [13]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: Yes	Mobile/edge: Yes; leakage control: No; external evaluation: No
SHRIMPAI [19]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: Yes; leakage control: No; external evaluation: No
Büyükarıkan [23]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: No; leakage control: No; external evaluation: No
Aung et al. [24]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: No; leakage control: No; external evaluation: No
Raj et al. [25]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: No; leakage control: No; external evaluation: No
ShrimpXNet [29]	Classification: Yes; instance segmentation: No	Robustness: Yes; XAI: Yes	Mobile/edge: No; leakage control: No; external evaluation: No
FeatherNetX [17]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: Yes	Mobile/edge: No; leakage control: No; external evaluation: Yes
Explainable ensemble [31]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: Yes	Mobile/edge: No; leakage control: No; external evaluation: No
IoT–ResNet system [27]	Classification: Yes; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: Yes; leakage control: No; external evaluation: No
Improved YOLO studies [20], [21]	Classification: No; instance segmentation: No	Robustness: No; XAI: No	Mobile/edge: No; leakage control: No; external evaluation: Yes

In Table 3.5, “No” means that the reviewed publication did not report or evaluate the corresponding component; it does not prove that the implementation could not support it. The coverage pattern is nevertheless clear. Classification is common, while mask-level instance segmentation, explicit leakage control, and external evaluation are uncommon. Robustness and XAI appear in a small number of recent studies, and mobile deployment is usually demonstrated in WSSV-specific or system-oriented work rather than in a four-class classification-plus-segmentation framework.

6. Gaps in the Literature/Technology

The first gap is the separation between image-level classification and disease-region localization. Existing shrimp studies predominantly predict a whole-image label or a bounding box. They rarely provide instance masks for BG and WSSV regions, and public datasets do not include independently reviewed mask annotations. As a result, classification confidence can be high without demonstrating that the model has localized visible disease evidence.

The second gap concerns evaluation validity. Most shrimp image papers report image-level splits without a detailed audit of repeated specimens, near duplicates, source overlap, or acquisition clusters. This omission is serious because several photographs can depict the same physical shrimp. A random split

may therefore measure recognition of specimen-specific appearance rather than generalization to unseen shrimp. The broader leakage literature shows that subject and site confounding can substantially inflate test performance [89–92].

The third gap is incomplete robustness evidence. Some systems use augmentation or adversarial training, but few evaluate a controlled set of task-appropriate corruptions and report class-balanced performance. The absence of such tests is especially important for mobile farm images, where illumination, focus, compression, background complexity, and camera characteristics can differ from the training set. Corruption results should still be bounded carefully because no finite benchmark can represent every farm and acquisition condition.

The fourth gap is weak comparability across studies. Reported metrics frequently use different disease spaces, datasets, image modalities, split policies, and averaging rules. Accuracy is often presented without Macro-F1 or classwise evidence, which can conceal poor minority-class behavior. Model size, latency, and deployment feasibility are also reported inconsistently. A defensible project must therefore establish controlled internal baselines rather than select a method from headline accuracy alone.

The fifth gap concerns reproducibility and artifact traceability. Several recent systems describe complex preprocessing, augmentation, ensembles, or optimization pipelines without releasing a complete executable package. When code, exact splits, class order, checkpoint-selection rules, and environment versions are unavailable, a reimplementation can legitimately differ from the paper result. Such differences must be analyzed through data protocol, preprocessing, augmentation, architecture, training, model selection, evaluation, and environment factors rather than attributed to one vague cause.

The sixth gap is the limited integration of research and application evidence. Mobile studies demonstrate useful interfaces but usually focus on binary WSSV screening, while multi-class studies often remain offline. Instance segmentation, robustness, XAI, Firebase-backed application workflows, and source-aware external evaluation are generally developed in separate publications. No reviewed system reports all of these components under a single traceable project protocol.

The seventh gap is annotation quality for disease-region segmentation. Producing masks requires both identifying the disease category and deciding where visible evidence begins and ends. This is substantially harder than assigning a whole-image classification label. Background removal may help classification by suppressing irrelevant context, but the same preprocessing can remove spatial or boundary information needed for segmentation. Single-annotator labeling improves internal consistency but leaves uncertainty because inter-annotator agreement and expert mask review are unavailable.

7. Justification for the Project

The literature justifies a project centered on lightweight four-class classification and BG/WSSV instance segmentation rather than object detection. Classification provides an efficient preliminary screening mechanism for Healthy, BG, WSSV, and co-infection images. Instance segmentation supplies a stricter, spatially explicit task that cannot be replaced by classification confidence or a bounding box. The two tasks therefore address different evidence requirements within the same application domain.

The classification direction is justified by the trade-off exposed in existing systems. High-capacity models and ensembles can report strong results but may be unsuitable for mobile inference, while compact systems can sacrifice class-balanced performance. Benchmarking representative TIMM and official YOLO classification families under one controlled seed and split provides an engineering basis for selecting a lightweight backbone. ShrimpXNet remains the closest published methodological reference, but the proposed classifier is evaluated as a direct improvement to the selected YOLO26m-cls engineering baseline rather than as a modification of ShrimpXNet.

The segmentation direction is justified by the scarcity of shrimp disease masks and the limitations of image-level explanation. Published attention and instance segmentation methods provide foundations, but the project-specific attention configuration and Fourier experiments are evaluated as proposed research. The Fourier study directly tests whether frequency-domain transformations improve robustness, small-region evidence, or instance segmentation performance across multiple configurations. Its conclusions must be based on the recorded experiments rather than on theoretical expectation alone.

Leakage-aware evaluation is justified by repeated-specimen risk. Grouping known views of the same shrimp protects the segmentation test set from one important source of information overlap. Source-wise splitting before multi-dataset merging similarly prevents images from being moved across partitions after combination. These measures do not guarantee universal generalization, but they provide a more credible estimate than an undocumented image-level random split.

Finally, mobile integration is justified as an application layer after the research methods are evaluated. TensorFlow Lite conversion, Android CPU inference, result and mask display, history management, Firebase services, and administrator functions demonstrate that selected models can participate in a usable workflow. The application remains a preliminary screening prototype: it does not replace laboratory diagnosis, veterinary assessment, treatment prescription, or formal farmer user-acceptance testing.

Taken together, the project is necessary because the reviewed literature rarely combines lightweight classification, class-imbalance handling, robustness testing, XAI, mask-level disease localization, leakage-aware splitting, multi-source evaluation, reproducibility artifacts, and mobile deployment. The intended contribution is not to claim superiority over incomparable published percentages. It is to construct and evaluate a traceable research pipeline in which each methodological claim is tied to a controlled baseline, a documented dataset protocol, and explicit limitations.

References

1. T. W. Flegel, ”Major viral diseases of the black tiger prawn (Penaeus monodon) in Thailand,” World Journal of Microbiology and Biotechnology, vol. 13, pp. 433–442, 1997, doi: 10.1023/A:1018580301578.
2. D. V. Lightner, ”Strategies for the control of viral diseases of shrimp in the Americas,” Fish Pathology, vol. 33, no. 4, pp. 165–180, 1998, doi: 10.3147/jsfp.33.165.
3. P. J. Walker and C. V. Mohan, ”Viral disease emergence in shrimp aquaculture: Origins, impact and the effectiveness of health management strategies,” Reviews in Aquaculture, vol. 1, no. 2, pp. 125–154, 2009, doi: 10.1111/j.1753-5131.2009.01007.x.
4. D. V. Lightner, ”Virus diseases of farmed shrimp in the Western Hemisphere (the Americas): A review,” Journal of Invertebrate Pathology, vol. 106, no. 1, pp. 110–130, 2011, doi: 10.1016/j.jip.2010.09.012.
5. G. D. Stentiford et al., ”Disease will limit future food supply from the global crustacean fishery and aquaculture sectors,” Journal of Invertebrate Pathology, vol. 110, no. 2, pp. 141–157, 2012, doi: 10.1016/j.jip.2012.03.013.
6. T. W. Flegel, ”Historic emergence, impact and current status of shrimp pathogens in Asia,” Journal of Invertebrate Pathology, vol. 110, no. 2, pp. 166–173, 2012, doi: 10.1016/j.jip.2012.03.004.
7. C. H. Seibert and A. R. Pinto, ”Challenges in shrimp aquaculture due to viral diseases: Distribution and biology of the five major penaeid viruses and interventions to avoid viral incidence and dispersion,” Brazilian Journal of Microbiology, vol. 43, no. 3, pp. 857–864, 2012, doi: 10.1590/S1517-83822012000300002.
8. S. Thitamadee et al., ”Review of current disease threats for cultivated penaeid shrimp in Asia,” Aquaculture, vol. 452, pp. 69–87, 2016, doi: 10.1016/j.aquaculture.2015.10.028.
9. A. P. Shinn et al., ”Asian shrimp production and the economic costs of disease,” Asian Fisheries Science, vol. 31, suppl. 1, pp. 29–58, 2018, doi: 10.33997/j.afs.2018.31.S1.003.
10. X. Yang, S. Zhang, J. Liu, Q. Gao, S. Dong, and C. Zhou, ”Deep learning for smart fish farming: Applications, opportunities and challenges,” Reviews in Aquaculture, vol. 13, no. 1, pp. 66–90, 2021, doi: 10.1111/raq.12464.
11. N. Duong-Trung, L. D. Quach, and C. N. Nguyen, ”Towards classification of shrimp diseases using transferred convolutional neural networks,” Advances in Science, Technology and Engineering Systems Journal, vol. 5, no. 4, pp. 724–732, 2020, doi: 10.25046/aj050486.
12. Q. Wang, C. Qian, P. Nie, and M. Ye, ”Rapid detection of Penaeus vannamei diseases via an improved LeNet,” Aquacultural Engineering, vol. 100, art. 102296, 2023, doi: 10.1016/j.aquaeng.2022.102296.
13. L. S. Querol, M. O. Cordel II, D. J. A. Rustia, and M. N. M. Santos, ”Application for White Spot Syndrome Virus (WSSV) monitoring using edge machine learning,” arXiv:2308.04151, 2023, doi: 10.48550/arXiv.2308.04151. [Preprint].
14. M. M. Islam et al., ”ShrimpDiseaseBD: An image dataset for detecting shrimp diseases in the aquaculture sector of Bangladesh,” Data in Brief, vol. 60, art. 111553, 2025, doi: 10.1016/j.dib.2025.111553.
15. M. M. Islam, A. Sarker, A. Choudhury, N. Ahmed, and A. A. S. R. Rasel, ”ShrimpDiseaseImageBD: An image dataset for computer vision-based detection of shrimp diseases in Bangladesh,” Mendeley Data, Version 3, 2025, doi: 10.17632/jhrtdj9txm.3.
16. S. I. Ahmed and D. M. Farid, ”TigerShrimpBD: A tiger shrimp image dataset,” Mendeley Data, Version 1, 2025, doi: 10.17632/9dj4sk5d55.1.
17. S. Sharma et al., ”A lightweight deep learning architecture for automatic shrimp disease classification,” Scientific Reports, vol. 16, art. 13837, 2026, doi: 10.1038/s41598-026-44195-z.
18. M. S. Ahmed and S. M. Jeba, ”SalmonScan: A novel image dataset for machine learning and deep learning analysis in fish disease detection in aquaculture,” Data in Brief, vol. 54, art. 110388, 2024, doi: 10.1016/j.dib.2024.110388.
19. M. S. Eder, K. C. Ampolitod, N. H. Bolao, R. J. A. Pajal, and E. M. D. Racho, ”SHRIMPAI: A mobile application for the early detection of white spot syndrome virus in shrimp using convolutional

neural network," Mindanao Journal of Science and Technology, vol. 22, no. 2, pp. 196–220, 2024, doi: 10.61310/mjst.v22i2.2189.

20. C. Xu et al., "Shrimp diseases detection method based on improved YOLOv8 and multiple features," Smart Agriculture, vol. 6, no. 2, pp. 62–71, 2024, doi: 10.12133/j.smartag.SA201311014.
21. Y. Fei, G. Wang, F. Liu, R. Zang, X. Sun, and H. Chang, "Lightweight shrimp disease detection research based on YOLOv8n," arXiv:2507.02354, 2025, doi: 10.48550/arXiv.2507.02354. [Preprint].
22. L. Ramachandran, S. P. Mangaiyarkarasi, A. Subramanian, and S. Senthilkumar, "Shrimp classification for white spot syndrome detection through enhanced gated recurrent unit-based wild geese migration optimization algorithm," Virus Genes, vol. 60, no. 2, pp. 134–147, 2024, doi: 10.1007/s11262-023-02049-0.
23. B. Büyükarıkan, "Robust shrimp disease detection using multi-model convolutional neural networks-based ensemble strategies," Aquacultural Engineering, vol. 111, art. 102616, 2025, doi: 10.1016/j.aquaeng.2025.102616.
24. T. Aung, R. Vanichviriyakit, K. Chayantrakom, S. Amornsamankul, and P. Huabsomboon, "CNN-based identification of pathogens of concern in shrimp," Animals, vol. 15, no. 21, art. 3194, 2025, doi: 10.3390/ani15213194.
25. A. S. Raj et al., "Enhanced recurrent capsule network with hybrid optimization model for shrimp disease detection," Scientific Reports, vol. 15, art. 10400, 2025, doi: 10.1038/s41598-025-94413-3.
26. P. Kanakamedala, V. M. Sunkara, J. R. Kumar, M. B. Reddy, and D. D. Prasad, "Shrimp disease classification and medicine recommendation using deep learning techniques," in Proc. International Conference on Innovations in Intelligent Systems: Advancements in Computing, Communication, and Cybersecurity (ISAC3), 2025, pp. 1–6, doi: 10.1109/ISAC364032.2025.11156739.
27. T. D. Le, N. M. Tu, H. K. Tu, and N. H. Quan, "An integrated IoT and AI monitoring system for early shrimp disease detection in Vietnam," VNUHCM Journal of Science and Technology Development, vol. 29, no. 1, pp. 3982–3993, 2026, doi: 10.32508/stdj.v29i1.4459.
28. P. Lou, Y. Zhao, S. Ma, J. Zhao, Y. Zheng, and J. Li, "An assessment method for the severity of shrimp red-leg disease based on E-DarkNet53 and multichannel feature fusion deep learning network," Smart Agricultural Technology, vol. 14, art. 101855, 2026, doi: 10.1016/j.atech.2026.101855.
29. I. H. Jone, D. M. R. B. Masud, P. Sarker, S. F. A. Labib, N. Islam, and F. Billah, "ShrimpXNet: A transfer learning framework for shrimp disease classification with augmented regularization, adversarial training, and explainable AI," arXiv:2601.00832, 2026, doi: 10.48550/arXiv.2601.00832. [Preprint].
30. F. Azhar, M. Syahrir, S. Alim, and Rismayanti, "Innovation of early detection system for Vannamei shrimp disease using integration of MobileNet-V3 architecture and K-nearest neighbors algorithm," Ingénierie des Systèmes d'Information, vol. 31, no. 4, pp. 1117–1126, 2026, doi: 10.18280/isi.310409.
31. N. M. Khiem, H. T. Kiet, and L. H. Q. Bao, "An explainable ensemble CNN approach for automated shrimp disease diagnosis using enhanced aquaculture image data," in Advances and Trends in Artificial Intelligence: IEA/AIE 2026, Lecture Notes in Computer Science, vol. 16615, Singapore: Springer, 2026, pp. 82–87, doi: 10.1007/978-981-92-2891-1_7.
32. T. E. George and E. Mathakari, "Shrimp disease detection using deep learning techniques," AIP Conference Proceedings, vol. 3344, no. 1, art. 020003, 2026, doi: 10.1063/5.0317063.
33. J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2015, pp. 3431–3440.
34. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention, Lecture Notes in Computer Science, vol. 9351, 2015, pp. 234–241, doi: 10.1007/978-3319-24574-4_28.
35. K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in Proc. IEEE Int. Conf. Computer Vision, 2017, pp. 2961–2969, doi: 10.1109/ICCV.2017.322.
36. D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-time instance segmentation," in Proc. IEEE/CVF Int. Conf. Computer Vision, 2019, pp. 9157–9166, doi: 10.1109/ICCV.2019.00925.
37. A. Kirillov, Y. Wu, K. He, and R. Girshick, "PointRend: Image segmentation as rendering," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2020, pp. 9799–9808.
38. Z. Tian, C. Shen, and H. Chen, "Conditional convolutions for instance segmentation," in Proc. European Conf. Computer Vision, 2020, pp. 282–298.

39. X. Wang et al., "SOLOv2: Dynamic and fast instance segmentation," in Advances in Neural Information Processing Systems 33, 2020.
40. B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2022, pp. 1290–1299.
41. H. Liu et al., "YolactEdge: Real-time instance segmentation on the edge," in Proc. IEEE Int. Conf. Robotics and Automation, 2021, pp. 9579–9585, doi: 10.1109/ICRA48506.2021.9561858.
42. J. He et al., "FastInst: A simple query-based model for real-time instance segmentation," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2023, pp. 23663–23672.
43. R. Zhang et al., "MobileInst: Video instance segmentation on the mobile," in Proc. AAAI Conf. Artificial Intelligence, vol. 38, no. 7, 2024, pp. 7155–7163.
44. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems 25, 2012, pp. 1097–1105.
45. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in Proc. ICLR, 2015.
46. C. Szegedy et al., "Going deeper with convolutions," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2015, pp. 1–9, doi: 10.1109/CVPR.2015.7298594.
47. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
48. G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2017, pp. 4700–4708, doi: 10.1109/CVPR.2017.243.
49. F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2017, pp. 1251–1258, doi: 10.1109/CVPR.2017.195.
50. F. N. Iandola et al., "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size," in Proc. ICLR, 2017.
51. A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.
52. X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 6848–6856, doi: 10.1109/CVPR.2018.00716.
53. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.
54. M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2019, pp. 2820–2828.
55. A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Computer Vision, 2019, pp. 1314–1324, doi: 10.1109/ICCV.2019.00140.
56. M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. 36th Int. Conf. Machine Learning, 2019, pp. 6105–6114.
57. K. Han et al., "GhostNet: More features from cheap operations," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2020, pp. 1580–1589.
58. X. Ding et al., "RepVGG: Making VGG-style ConvNets great again," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2021, pp. 13733–13742.
59. M. Tan and Q. Le, "EfficientNetV2: Smaller models and faster training," in Proc. 38th Int. Conf. Machine Learning, 2021, pp. 10096–10106.
60. S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," in Proc. ICLR, 2022.
61. Z. Liu et al., "A ConvNet for the 2020s," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2022, pp. 11976–11986.
62. M. Maaz et al., "EdgeNeXt: Efficiently amalgamated CNN-transformer architecture for mobile vision applications," in ECCV Workshops, Lecture Notes in Computer Science, vol. 13802, 2022, pp. 3–20.

63. P. Vasu et al., ”MobileOne: An improved one millisecond mobile backbone,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2023, pp. 7907–7917.
64. P. Vasu et al., ”FastViT: A fast hybrid vision transformer using structural reparameterization,” in Proc. IEEE/CVF Int. Conf. Computer Vision, 2023, pp. 5785–5795.
65. D. Qin et al., ”MobileNetV4: Universal models for the mobile ecosystem,” in Proc. European Conf. Computer Vision, 2024.
66. T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, ”Focal loss for dense object detection,” in Proc. IEEE Int. Conf. Computer Vision, 2017, pp. 2980–2988.
67. Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, ”Class-balanced loss based on effective number of samples,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2019, pp. 9268–9277.
68. K. Cao, C. Wei, A. Gaidon, N. Aréchiga, and T. Ma, ”Learning imbalanced datasets with label-distribution-aware margin loss,” in Advances in Neural Information Processing Systems 32, 2019.
69. E. Ben-Baruch, T. Ridnik, N. Zamir, A. Noy, I. Friedman, M. Protter, and L. Zelnik-Manor, ”Asymmetric loss for multi-label classification,” in Proc. IEEE/CVF Int. Conf. Computer Vision, 2021, pp. 82–91.
70. J. Hu, L. Shen, and G. Sun, ”Squeeze-and-excitation networks,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 7132–7141.
71. S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, ”CBAM: Convolutional block attention module,” in Proc. European Conf. Computer Vision, 2018, pp. 3–19.
72. Q. Wang et al., ”ECA-Net: Efficient channel attention for deep convolutional neural networks,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2020, pp. 11534–11542.
73. Q. Hou, D. Zhou, and J. Feng, ”Coordinate attention for efficient mobile network design,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2021, pp. 13713–13722.
74. L. Yang, R.-Y. Zhang, L. Li, and X. Xie, ”SimAM: A simple, parameter-free attention module for convolutional neural networks,” in Proc. 38th Int. Conf. Machine Learning, 2021, pp. 11863–11874.
75. D. Misra, T. Nalamada, A. U. Arasanipalai, and Q. Hou, ”Rotate to attend: Convolutional triplet attention module,” in Proc. IEEE Winter Conf. Applications of Computer Vision, 2021, pp. 3139–3148.
76. D. Yin, R. G. Lopes, J. Shlens, E. D. Cubuk, and J. Gilmer, ”A Fourier perspective on model robustness in computer vision,” in Advances in Neural Information Processing Systems 32, 2019, pp. 13255–13265.
77. Y. Yang and S. Soatto, ”FDA: Fourier domain adaptation for semantic segmentation,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2020, pp. 4085–4095.
78. Q. Xu, R. Zhang, Y. Zhang, Y. Wang, and Q. Tian, ”A Fourier-based framework for domain generalization,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2021, pp. 14383–14392.
79. I. J. Goodfellow, J. Shlens, and C. Szegedy, ”Explaining and harnessing adversarial examples,” in Proc. ICLR, 2015.
80. H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, ”mixup: Beyond empirical risk minimization,” in Proc. ICLR, 2018.
81. D. Hendrycks and T. Dietterich, ”Benchmarking neural network robustness to common corruptions and perturbations,” in Proc. ICLR, 2019.
82. S. Yun et al., ”CutMix: Regularization strategy to train strong classifiers with localizable features,” in Proc. IEEE/CVF Int. Conf. Computer Vision, 2019, pp. 6023–6032.
83. D. Hendrycks et al., ”AugMix: A simple data processing method to improve robustness and uncertainty,” in Proc. ICLR, 2020.
84. B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, ”Learning deep features for discriminative localization,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2016, pp. 2921–2929.
85. R. R. Selvaraju et al., ”Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in Proc. IEEE Int. Conf. Computer Vision, 2017, pp. 618–626.
86. A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, ”Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks,” in Proc. IEEE Winter Conf. Ap-plications of Computer Vision, 2018, pp. 839–847.

87. H. Wang et al., ”Score-CAM: Score-weighted visual explanations for convolutional neural networks,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops, 2020, pp. 24–25.
88. M. B. Muhammad and M. Yeasin, ”Eigen-CAM: Class activation map using principal components,” in Proc. Int. Joint Conf. Neural Networks, 2020, pp. 1–7, doi: 10.1109/IJCNN48605.2020.9206626.
89. S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, ”Leakage in data mining: Formulation, detection, and avoidance,” ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, art. 15, 2012, doi: 10.1145/2382577.2382579.
90. J. R. Zech et al., ”Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study,” PLoS Medicine, vol. 15, no. 11, e1002683, 2018, doi: 10.1371/journal.pmed.1002683.
91. M. Roberts et al., ”Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans,” Nature Machine Intelligence, vol. 3, pp. 199–217, 2021, doi: 10.1038/s42256-021-00307-0.
92. I. E. Tampu, A. Eklund, and N. Haj-Hosseini, ”Inflation of test accuracy due to data leakage in deep learning-based classification of OCT images,” Scientific Data, vol. 9, art. 580, 2022, doi: 10.1038/s41597-022-01618-6.
93. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, ”You only look once: Unified, real-time object detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2016, pp. 779–788.
94. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, ”YOLOv10: Real-time end-to-end object detection,” in Advances in Neural Information Processing Systems 37, 2024.
95. G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, ”Ultralytics YOLO26: Unified real-time end-to-end vision models,” arXiv:2606.03748, 2026, doi: 10.48550/arXiv.2606.03748. [Preprint].